Análise dos Repasses da Arrecadação Federal

Problemas e Fontes

Contexto

> **A arrecadação federal movimenta trilhões de reais todos os anos, mas depois que esses recursos são arrecadados, para onde parte desse dinheiro é destinada?**

Dentro desse universo, existe uma categoria específica de recursos: as **contribuições destinadas a outras entidades e fundos**, conhecidos como **"terceiros"**.

Esses valores são arrecadados por meio de documentos como:

- **DARF** — Documento de Arrecadação de Receitas Federais
- **GPS** — Guia da Previdência Social

Objetivo da análise

Neste estudo, vamos analisar especificamente esses **repasses**, buscando entender:

- **Quanto** é destinado a essas entidades;
- **Quais instituições** recebem os maiores valores;
- **Como os repasses evoluem** ao longo do tempo;
- **Qual a representatividade financeira** desses repasses em relação à arrecadação federal;
- **Quais instituições possuem maior relevância financeira** dentro dessa categoria.

Dados utilizados

Para esta análise, serão utilizados **dados públicos catalogados pelo Governo Federal em 05 de agosto de 2022**.

Com um pouco mais de 2000 registros. Esse CSV carrega algumas informações sobre data, valores e informações da entidade.

Informação	Período
Início dos registros	**Janeiro de 2015**
Último registro	**Agosto de 2026**
Catalogação	**05 de agosto de 2022**
Categoria	**Outras entidades e fundos — "Terceiros"**

A partir desses dados, será possível analisar tanto a **distribuição dos recursos entre as entidades** quanto sua **evolução ao longo do tempo**, identificando quais instituições apresentam maior relevância financeira dentro dessa categoria de repasses.

O dado que vamos utilizar está armazenado em um link do drive, porém, foi retirado exatamente no link abaixo.

link: https://dados.gov.br/dados/conjuntos-dados/repasses-da-arrecadacao-federal

Ingestão dos dados

###Importando o arquivo

In [1]:
```
import pandas as pd
import re

link =
"https://drive.google.com/file/d/1lLxFUzdouLUESomjn0tqO_jjOiG2V_UT/view?
usp=sharing"

file_id = re.search(r"/d/([a-zA-Z0-9_-]+)", link).group(1)

url = f"https://drive.google.com/uc?id={file_id}"

df = pd.read_csv(url, sep=";")

df.head()
```

Out[1]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
0	jan/15	FNDE	153173 / 15253	2.604.077.590,00
1	jan/15	INCRA	373001 / 37201	181.456.016,00
2	jan/15	SENAI	33.564.543/0001-90	219.119.330,00
3	jan/15	SESI	33.641.358/0001-52	313.370.139,00
4	jan/15	SENAC	33.469.172/0001-68	334.005.249,00

Qualidade dos dados

###Quantidade de linhas

In [2]:
```
df.count()
```

Out[2]:

	0
Mês / Ano de Referência	2380
Entidade	2380
UC/CNPJ	2380
Total Repassado	2380

dtype: int64

###Valores nulos

In [3]:
```
df_linhas_com_nulos = df[df.isna().any(axis=1)]
df_linhas_com_nulos
```

Out[3]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2380	NaN	NaN	NaN	NaN
2381	NaN	NaN	NaN	NaN
2382	NaN	NaN	NaN	NaN
2383	NaN	NaN	NaN	NaN
2384	NaN	NaN	NaN	NaN
2385	NaN	NaN	NaN	NaN
2386	NaN	NaN	NaN	NaN
2387	NaN	NaN	NaN	NaN
2388	NaN	NaN	NaN	NaN
2389	NaN	NaN	NaN	NaN
2390	NaN	NaN	NaN	NaN
2391	NaN	NaN	NaN	NaN
2392	NaN	NaN	NaN	NaN
2393	NaN	NaN	NaN	NaN
2394	NaN	NaN	NaN	NaN
2395	NaN	NaN	NaN	NaN
2396	NaN	NaN	NaN	NaN

###Registros duplicados

In [4]:
```
print("Linhas idênticas:", df.duplicated().sum())
chave = ["Mês / Ano de Referência", "Entidade"]
dup = df[df.duplicated(subset=chave, keep=False)]
dup.sort_values(chave)
```

```
Linhas idênticas: 16
```

Out[4]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2380	NaN	NaN	NaN	NaN
2381	NaN	NaN	NaN	NaN
2382	NaN	NaN	NaN	NaN
2383	NaN	NaN	NaN	NaN
2384	NaN	NaN	NaN	NaN
2385	NaN	NaN	NaN	NaN
2386	NaN	NaN	NaN	NaN
2387	NaN	NaN	NaN	NaN
2388	NaN	NaN	NaN	NaN
2389	NaN	NaN	NaN	NaN
2390	NaN	NaN	NaN	NaN
2391	NaN	NaN	NaN	NaN
2392	NaN	NaN	NaN	NaN
2393	NaN	NaN	NaN	NaN
2394	NaN	NaN	NaN	NaN
2395	NaN	NaN	NaN	NaN
2396	NaN	NaN	NaN	NaN

###Tipos de dados incorretos

In [5]:
```
base = df.dropna()
base.sample(8, random_state=1)
```

Out[5]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
877	abr/19	SENAR	37.138.245/0001-90	89.251.005,00
2086	mar/25	SENAT	73.471.963/0001-47	48.414.953,47
120	ago/15	INCRA	373001 / 37201	99.312.699,00
2266	fev/26	SESC	33.469.164/0001-11	785526796,3
107	jul/15	SESC	33.469.164/0001-11	357.769.318,00
1232	jan/21	DPC	852001 / 00001	21.364.259,00
1264	mar/21	SEBRAE	00.330.854/0001-45	293.937.886,00
1684	abr/23	INCRA	373001 / 37201	196.583.829,00

###Formatos inconsistentes

In [6]:
```
com_milhar = base["Total Repassado"].str.contains(r"\.\d{3}")
sem_milhar = ~com_milhar & (base["Total Repassado"] != "0")
print("Valores com separador de milhar:", com_milhar.sum())
print("Valores sem separador de milhar:", sem_milhar.sum())
base.loc[sem_milhar, "Mês / Ano de Referência"].unique()
```

```
Valores com separador de milhar: 2070
Valores sem separador de milhar: 176
```

Out[6]:
```
array(['out/25', 'nov/25', 'dez/25', 'jan/26', 'fev/26', 'mar/26',
       'abr/26', 'mai/26', 'jun/26', 'jul/26', 'ago/26'], dtype=object)
```

In [7]:
```
base[sem_milhar].head(10)
```

Out[7]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2193	out/25	FNDE	153173 / 15253	2950662651
2194	out/25	INCRA	373001 / 37201	252386984,9
2195	out/25	SENAI	33.564.543/0001-90	218030840,1
2196	out/25	SESI	33.641.358/0001-52	319516639,5
2197	out/25	SENAC	33.469.172/0001-68	429318140,3
2198	out/25	SESC	33.469.164/0001-11	739128752,9
2199	out/25	SEBRAE	00.330.854/0001-45	472771291,6
2201	out/25	DPC	852001 / 00001	23184616,26
2202	out/25	ANAC	122002 / 00001	24891430,8
2203	out/25	SENAR	37.138.245/0001-90	215406046,8

In [8]:
```
[repr(e) for e in base["Entidade"].unique()]
```

Out[8]:
```
["'FNDE'",
 "'INCRA'",
 "'SENAI'",
 "'SESI'",
 "'SENAC'",
 "'SESC'",
 "'SEBRAE'",
 "'EMBRATUR'",
 "'DPC'",
 "'ANAC'",
 "'SENAR'",
 "'SEST'",
 "'SENAT'",
 "'SESCOOP'",
 "'SDR'",
 "'APEX-BR'",
 "'ABDI'",
 "'FNDE '",
 "'APEX'"]
```

In [9]:
```
base["Entidade"].value_counts()
```

Out[9]:

Entidade	count
INCRA	140
SENAI	140
SESI	140
SEST	140
SENAC	140
SESC	140
SEBRAE	140
EMBRATUR	140
DPC	140
ANAC	140
SENAR	140
ABDI	140
SENAT	140
SESCOOP	140
SDR	140
FNDE	137
APEX-BR	132
APEX	8
FNDE	3

dtype: int64

In [10]:
```
apex = base[base["Entidade"].isin(["APEX", "APEX-BR"])]
apex.groupby("Entidade").agg(
    primeiro_mes=("Mês / Ano de Referência", "first"),
    ultimo_mes=("Mês / Ano de Referência", "last"),
    registros=("Mês / Ano de Referência", "count"),
    identificador=("UC/CNPJ", "unique"),
)
```

Out[10]:

Entidade	primeiro_mes	ultimo_mes	registros	identificador
APEX	jan/26	ago/26	8	[05.507.500/0001-38]
APEX-BR	jan/15	dez/25	132	[05.507.500/0001-38]

In [11]:
```
base.groupby(base["Entidade"].str.strip())["UC/CNPJ"].unique()
```

Out[11]:

Entidade	UC/CNPJ
ABDI	[07.200.966/0001-11]
ANAC	[122002 / 00001]
APEX	[05.507.500/0001-38]
APEX-BR	[05.507.500/0001-38]
DPC	[852001 / 00001]
EMBRATUR	[35.842.428/0001-66]
FNDE	[153173 / 15253]
INCRA	[373001 / 37201]
SDR	[420013 / 00001]
SEBRAE	[00.330.854/0001-45]
SENAC	[33.469.172/0001-68]
SENAI	[33.564.543/0001-90]
SENAR	[37.138.245/0001-90]
SENAT	[73.471.963/0001-47]
SESC	[33.469.164/0001-11]
SESCOOP	[03.087.543/0001-86]
SESI	[33.641.358/0001-52]
SEST	[73.471.989/0001-95]

dtype: object

###Valores inválidos ou impossíveis

In [12]:
```
valor = pd.to_numeric(
    base["Total Repassado"].str.replace(".", "",
regex=False).str.replace(",", ".", regex=False)
)
print("Valores negativos:", (valor < 0).sum())
print("Valores zerados:", (valor == 0).sum())
base[valor < 0]
```

```
Valores negativos: 1
Valores zerados: 134
```

Out[12]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2049	jan/25	ANAC	122002 / 00001	-318.467.181,81

In [13]:
```
base.loc[valor == 0, "Entidade"].value_counts()
```

Out[13]:

Entidade	count
EMBRATUR	134

dtype: int64

In [14]:
```
base[base["Entidade"] == "EMBRATUR"].assign(Valor=valor).query("Valor ≠ 0")
```

Out[14]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado	Valor
1027	jan/20	EMBRATUR	35.842.428/0001-66	90.202.735,00	90202735.0
1044	fev/20	EMBRATUR	35.842.428/0001-66	52.274.883,00	52274883.0
1061	mar/20	EMBRATUR	35.842.428/0001-66	50.715.821,00	50715821.0
1078	abr/20	EMBRATUR	35.842.428/0001-66	49.549.991,00	49549991.0
1095	mai/20	EMBRATUR	35.842.428/0001-66	45.933.122,00	45933122.0
1112	jun/20	EMBRATUR	35.842.428/0001-66	48.627.667,00	48627667.0

###Possíveis outliers

In [15]:
```
base = base.assign(Entidade=base["Entidade"].str.strip(), Valor=valor)

def fora_iqr(s):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)

outlier = base.groupby("Entidade")["Valor"].transform(fora_iqr)
print("Possíveis outliers:", outlier.sum())
base[outlier].groupby("Entidade").size()
```

```
Possíveis outliers: 52
```

Out[15]:

Entidade	0
ABDI	4
ANAC	8
APEX	1
APEX-BR	3
DPC	2
EMBRATUR	6
FNDE	3
INCRA	2
SEBRAE	3
SENAC	3
SENAI	4
SENAT	2
SESC	3
SESCOOP	2
SESI	4
SEST	2

dtype: int64

In [16]:
```
base.loc[outlier, "Mês / Ano de Referência"].str[:3].value_counts()
```

Out[16]:

Mês / Ano de Referência	count
jan	40
jul	3
mai	2
ago	2
fev	1
abr	1
mar	1
jun	1
nov	1

dtype: int64

In [17]:
```
base[outlier].sort_values("Valor", ascending=False).head(10)
```

Out[17]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado	Valor
2244	jan/26	FNDE	153173 / 15253	5146437727	5.146438e+09
2040	jan/25	FNDE	153173 / 15253	4.785.575.662,21	4.785576e+09
1836	jan/24	FNDE	153173 / 15253	4.322.353.057,00	4.322353e+09
2249	jan/26	SESC	33.469.164/0001-11	1332041212	1.332041e+09
2045	jan/25	SESC	33.469.164/0001-11	1.222.040.575,77	1.222041e+09
1841	jan/24	SESC	33.469.164/0001-11	1.087.903.211,00	1.087903e+09
2251	jan/26	SEBRAE	00.330.854/0001-45	858527955	8.585280e+08
2046	jan/25	SEBRAE	00.330.854/0001-45	798.270.386,80	7.982704e+08
2248	jan/26	SENAC	33.469.172/0001-68	774741173,2	7.747412e+08
2044	jan/25	SENAC	33.469.172/0001-68	705.641.786,36	7.056418e+08

Limpeza e preparação dos dados

###Removendo valores nulos

In [18]:
```
df = df.dropna().reset_index(drop=True)
df.count()
```

Out[18]:

	0
Mês / Ano de Referência	2380
Entidade	2380
UC/CNPJ	2380
Total Repassado	2380

dtype: int64

###Removendo registros duplicados

As 16 linhas idênticas encontradas eram as linhas vazias. Após a remoção não restam duplicados.

In [19]:
```
df = df.drop_duplicates()
print("Linhas idênticas:", df.duplicated().sum())
print("Duplicados por mês e entidade:", df.duplicated(subset=["Mês / Ano de Referência", "Entidade"]).sum())
```

```
Linhas idênticas: 0
Duplicados por mês e entidade: 0
```

###Padronizando as entidades

In [20]:
```
df["Entidade"] = df["Entidade"].str.strip().replace({"APEX": "APEX-BR"})
df["Entidade"].value_counts()
```

Out[20]:

Entidade	count
FNDE	140
INCRA	140
SENAI	140
SESI	140
SENAC	140
SESC	140
SEBRAE	140
EMBRATUR	140
DPC	140
ANAC	140
SENAR	140
SEST	140
SENAT	140
SESCOOP	140
SDR	140
APEX-BR	140
ABDI	140

dtype: int64

Foram removidos os espaços extras e APEX foi unificada como APEX-BR, já que as duas possuem o mesmo CNPJ e não se sobrepõem no tempo. Assim cada entidade passa a ter 140 registros, um por mês.

###Convertendo a data de referencia

In [21]:
```
meses = {"jan": 1, "fev": 2, "mar": 3, "abr": 4, "mai": 5, "jun": 6,
         "jul": 7, "ago": 8, "set": 9, "out": 10, "nov": 11, "dez": 12}
partes = df["Mês / Ano de Referência"].str.split("/", expand=True)
df["Mês / Ano de Referência"] = pd.to_datetime(
    pd.DataFrame({"year": 2000 + partes[1].astype(int), "month":
partes[0].map(meses), "day": 1})
)
df["Mês / Ano de Referência"].agg(["min", "max", "nunique"])
```

Out[21]:

	Mês / Ano de Referência
min	2015-01-01 00:00:00
max	2026-08-01 00:00:00
nunique	140

dtype: object

A data foi convertida para o primeiro dia do mês de referência, permitindo ordenação e análises temporais.

###Convertendo o valor repassado

In [22]:
```
df["Total Repassado"] = pd.to_numeric(
    df["Total Repassado"].str.replace(".", "", regex=False).str.replace(",",
".", regex=False)
)
pd.set_option("display.float_format", "{:,.2f}".format)
df.dtypes
```

Out[22]:

	0
Mês / Ano de Referência	datetime64[ns]
Entidade	object
UC/CNPJ	object
Total Repassado	float64

dtype: object

Os pontos de milhar foram removidos e a vírgula decimal trocada por ponto, o que funciona para os dois padrões encontrados. Os valores a partir de out/25 vieram com casas decimais truncadas na fonte; essa perda de precisão é menor que R$ 1 por registro em valores de milhões e não pode ser recuperada.

###Conjunto de dados resultante

In [23]:
```
df = df.sort_values(["Mês / Ano de Referência",
"Entidade"]).reset_index(drop=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2380 entries, 0 to 2379
Data columns (total 4 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Mês / Ano de Referência  2380 non-null   datetime64[ns]
 1   Entidade                 2380 non-null   object
 2   UC/CNPJ                  2380 non-null   object
 3   Total Repassado          2380 non-null   float64
dtypes: datetime64[ns](1), float64(1), object(2)
memory usage: 74.5+ KB
```

In [24]:
```
assert df.isna().sum().sum() == 0
assert not df.duplicated().any()
assert not df.duplicated(subset=["Mês / Ano de Referência",
"Entidade"]).any()
assert df["Entidade"].nunique() == 17
assert df.groupby("Entidade").size().eq(140).all()
assert df.groupby("Entidade")["UC/CNPJ"].nunique().eq(1).all()
assert df["Mês / Ano de Referência"].nunique() == 140
assert pd.api.types.is_datetime64_any_dtype(df["Mês / Ano de Referência"])
assert pd.api.types.is_float_dtype(df["Total Repassado"])

print("Registros:", len(df))
print("Entidades:", df["Entidade"].nunique())
print("Período:", df["Mês / Ano de Referência"].min().strftime("%m/%Y"),
"a", df["Mês / Ano de Referência"].max().strftime("%m/%Y"))
```

```
Registros: 2380
Entidades: 17
Período: 01/2015 a 08/2026
```

In [25]:
```
df["Total Repassado"].describe()
```

Out[25]:

	Total Repassado
count	2,380.00
mean	248,313,744.98
std	519,683,030.06
min	-318,467,181.81
25%	19,802,095.50
50%	66,269,871.63
75%	235,194,962.90
max	5,146,437,727.00

dtype: float64

In [26]:
```
df.to_csv("repasses_arrecadacao_federal_limpo.csv", sep=";", index=False)
df.head(17)
```

Out[26]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
0	2015-01-01	ABDI	07.200.966/0001-11	8,917,017.00
1	2015-01-01	ANAC	122002 / 00001	30,221,162.00
2	2015-01-01	APEX-BR	05.507.500/0001-38	62,241,408.00
3	2015-01-01	DPC	852001 / 00001	20,988,896.00
4	2015-01-01	EMBRATUR	35.842.428/0001-66	0.00
5	2015-01-01	FNDE	153173 / 15253	2,604,077,590.00
6	2015-01-01	INCRA	373001 / 37201	181,456,016.00
7	2015-01-01	SDR	420013 / 00001	32,001,111.00
8	2015-01-01	SEBRAE	00.330.854/0001-45	428,247,534.00
9	2015-01-01	SENAC	33.469.172/0001-68	334,005,249.00
10	2015-01-01	SENAI	33.564.543/0001-90	219,119,330.00
11	2015-01-01	SENAR	37.138.245/0001-90	40,999,819.00
12	2015-01-01	SENAT	73.471.963/0001-47	37,293,612.00
13	2015-01-01	SESC	33.469.164/0001-11	597,447,338.00
14	2015-01-01	SESCOOP	03.087.543/0001-86	39,616,737.00
15	2015-01-01	SESI	33.641.358/0001-52	313,370,139.00
16	2015-01-01	SEST	73.471.989/0001-95	57,041,322.00

O conjunto final possui 2.380 registros (17 entidades × 140 meses, de jan/2015 a ago/2026), sem valores ausentes, sem duplicados, com datas e valores em tipos adequados e nomes de entidades padronizados. Ele foi salvo em repasses_arrecadacao_federal_limpo.csv e será a base para as próximas etapas do projeto.